

AgentGate Technical Foundation Document

Sprint 0 – Data Engineering Foundation

Project: AgentGate
Phase: Sprint 0 – Kick-off & Project Initiation
Owner: Data Engineer
Version: 1.0
Date: June 2026

1. Purpose

Dokumen ini mendefinisikan fondasi teknis awal untuk AgentGate MVP, meliputi:

- Repository Structure
- Local Development Setup
- Database Choice
- Connector & Authentication Storage Strategy
- Environment Strategy

Dokumen ini menjadi referensi bersama untuk seluruh tim selama fase pengembangan AgentGate.

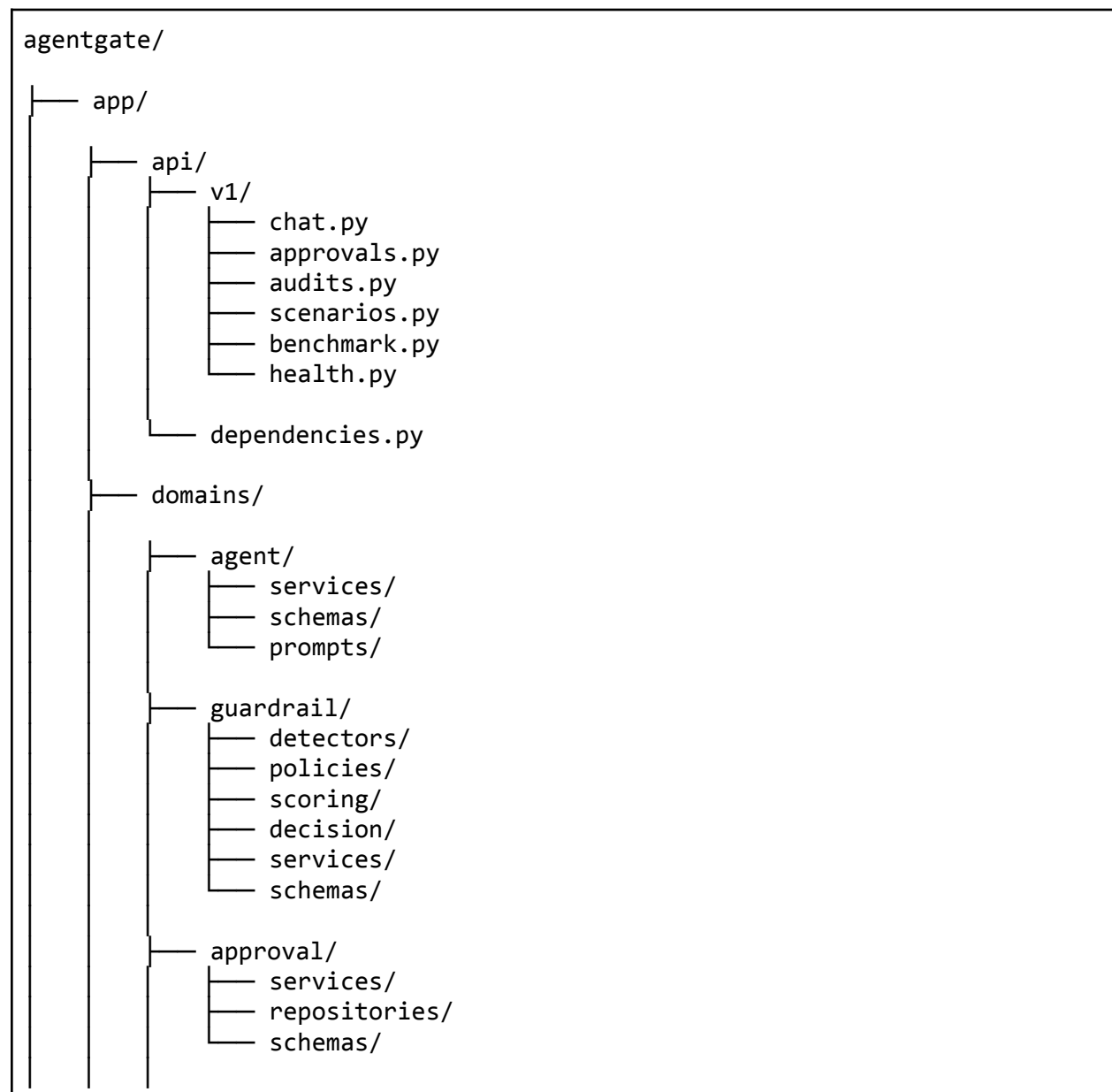
2. Architecture Decision Summary

Area	Decision
Architecture Style	Modular Monolith
Backend Framework	FastAPI
Programming Language	Python 3.11
Database	PostgreSQL
ORM	SQLAlchemy
Migration Tool	Alembic
Browser Automation	Playwright
Queue Layer	Redis (future-ready)
Containerization	Docker & Docker Compose
Secret Management (MVP)	.env

Area	Decision
Secret Management (Production)	AWS Secrets Manager / Hashicorp Vault

3. Repository Structure

AgentGate menggunakan pendekatan Modular Monolith untuk menjaga kesederhanaan MVP sekaligus memungkinkan ekstraksi domain menjadi microservices di masa depan.



```

├── audit/
│   ├── services/
│   ├── repositories/
│   └── schemas/
├── connector/
│   ├── gmail/
│   ├── github/
│   ├── calendar/
│   ├── telegram/
│   ├── stripe/
│   └── filesystem/
├── browser/
│   ├── snapshot/
│   ├── playwright/
│   └── selector_map/
├── benchmark/
│   ├── services/
│   └── reports/
├── executors/
│   ├── api_executor.py
│   ├── browser_executor.py
│   └── decision_router.py
├── llm/
│   ├── providers/
│   │   ├── openai.py
│   │   ├── anthropic.py
│   │   └── mock.py
│   ├── tool_registry.py
│   └── planner.py
├── database/
│   ├── session.py
│   ├── base.py
│   └── migrations/
├── models/
│   ├── approval.py
│   ├── audit_log.py
│   ├── action_request.py
│   └── execution.py
├── workers/
│   └── audit_worker.py

```

```
├── benchmark_worker.py
├── connector_worker.py
├── config/
│   ├── settings.py
│   ├── logging.py
│   └── constants.py
├── utils/
│   ├── time.py
│   ├── hashing.py
│   └── validators.py
├── main.py
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └── fixtures/
├── scripts/
│   ├── seed_db.py
│   ├── benchmark.py
│   └── replay_scenario.py
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── connectors/
│   └── decisions/
├── deployment/
│   ├── docker/
│   ├── compose/
│   └── nginx/
├── .env.example
├── docker-compose.yml
├── requirements.txt
├── pyproject.toml
└── README.md
```

4. Domain Boundaries

Agent Domain

Responsible for:

- Task orchestration
- Planner interaction
- Tool-call lifecycle
- ActionRequest creation

domains/agent

Guardrail Domain

Responsible for:

- Detectors
- Policy evaluation
- Risk scoring
- Decision generation

domains/guardrail

Approval Domain

Responsible for:

- Approval queue
- Approval workflow
- Human confirmation state

domains/approval

Audit Domain

Responsible for:

- Audit logging
- Decision history
- Execution trace storage

domains/audit

Connector Domain

Responsible for:

- Gmail API
- GitHub API
- Calendar API
- Telegram API
- Stripe Sandbox
- Local Filesystem

domains/connector

Browser Domain

Responsible for:

- Playwright automation
- Snapshot extraction
- Selector map generation

domains/browser

Benchmark Domain

Responsible for:

- Latency benchmarking
- Raw vs Guarded comparison
- Evaluation reports

domains/benchmark

5. Future Scalability Strategy

Current architecture follows Modular Monolith.

Future extraction candidates:

Domain	Future Service
audit	audit-service
approval	approval-service
connector	connector-service

Domain
benchmark

Future Service
benchmark-service

The following domains are expected to remain Python-based:

- agent
- guardrail
- browser
- llm

due to tight coupling with AI tooling and Playwright ecosystem.

6. Local Development Setup

Required Software

Backend

- Python 3.11

Database

- PostgreSQL 17

Container Runtime

- Docker
- Docker Compose

Browser Automation

- Playwright

Package Management

- pip / uv
-

Local Startup

Start Services

```
docker compose up -d
```

Run Backend

```
uvicorn app.main:app --reload
```

Run Database Migration

alembic upgrade head

7. Database Choice

Selected Database

PostgreSQL

Rationale

AgentGate requires storage for:

- Action Requests
- Audit Logs
- Approval Queue
- Execution History
- Benchmark Results

Many records contain semi-structured JSON payloads.

Example:

```
{  
  "action": "gmail_archive",  
  "risk_score": 0.82,  
  "decision": "NEED_APPROVAL"  
}
```

PostgreSQL JSONB support provides:

- Flexible schema
 - Efficient querying
 - Indexing support
 - Future scalability
-

8. Initial Data Model

action_requests

Stores proposed actions before execution.

Field	Type
id	UUID
action_type	VARCHAR
target_system	VARCHAR
payload	JSONB
status	VARCHAR
created_at	TIMESTAMP

audit_logs

Stores guardrail decisions.

Field	Type
id	UUID
action_id	UUID
decision	VARCHAR
risk_score	FLOAT
reasons	JSONB
created_at	TIMESTAMP

approvals

Stores approval workflow state.

Field	Type
id	UUID
action_id	UUID
status	VARCHAR
approved_by	VARCHAR
approved_at	TIMESTAMP

executions

Stores execution outcomes.

Field	Type
id	UUID
action_id	UUID

Field	Type
executor_type	VARCHAR
result	JSONB
created_at	TIMESTAMP

9. Connector Strategy

Connector Contract

All connectors must implement a common interface.

```
class BaseConnector:
```

```
    async def execute(
        self,
        action: str,
        payload: dict
    ):
        pass
```

Supported MVP Connectors

Gmail Connector

- Read Email
- Archive Email

GitHub Connector

- Read Repository
- Read File

Calendar Connector

- Read Events
- Create Events

Telegram Connector

- Send Messages

Stripe Sandbox Connector

- Sandbox Payment Operations

Filesystem Connector

- Read File
 - Write File
-

10. Authentication & Secret Storage Strategy

MVP

Secrets are stored using environment variables.

Example:

```
OPENAI_API_KEY=  
GITHUB_TOKEN=  
TELEGRAM_BOT_TOKEN=  
DATABASE_URL=
```

Secret Loading

Configuration is managed through:

`pydantic-settings`

Example:

```
class Settings(BaseSettings):  
    DATABASE_URL: str  
    OPENAI_API_KEY: str
```

Production

Future migration target:

- AWS Secrets Manager
- Hashicorp Vault

No secrets may be committed to source control.

11. Environment Strategy

Development

Purpose:

Local developer environment.

File:

`.env.dev`

Staging

Purpose:

Internal testing and demo validation.

File:

`.env.staging`

Production

Purpose:

Public deployment environment.

File:

`.env.prod`

12. Logging Strategy

All logs must be structured JSON logs.

Example:

```
{  
  "timestamp": "...",  
  "action_id": "...",  
  "decision": "NEED_APPROVAL",  
  "risk_score": 0.82  
}
```

Logging goals:

- Observability
 - Auditability
 - Benchmark analysis
-

13. Worker Strategy

Worker modules are introduced to support future asynchronous execution.

Initial workers:

workers/

```
audit_worker.py
benchmark_worker.py
connector_worker.py
```

Future architecture:

```
FastAPI
  ↓
Redis Queue
  ↓
Workers
```

This enables horizontal scaling without major architectural changes.

14. Sprint 0 Deliverables

The Data Engineering Sprint 0 is considered complete when the following artifacts are delivered:

- Repository Structure Definition
- Architecture Decision Record (ADR)
- Initial Database Design
- Local Development Setup Guide
- Connector Contract Definition
- Authentication Storage Plan
- Environment Strategy
- Future Scalability Plan

These artifacts serve as the technical foundation for Sprint 1 implementation.